CSP-203: Software Tools & Technologies Lab



Chapter – 6

Web Programming: Introduction to PHP & MySQL (Lecture-3)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Introduction
- ❑ Getting Started with PHP
- ❑ PHP Tags
- ❑ Comments, Whitespaces, Case Sensitivity
- ❑ PHP Variables and Naming
- ❑ Operators in PHP
- ❑ Decision Making Statement in PHP
- ❑ Loops in PHP
- ❑ Arrays in PHP
- ❑ PHP Web Concepts

Introduction

- **PHP** stands for **"PHP: Hypertext Preprocessor"** (recursive acronym).
- Created by **Rasmus Lerdorf** in **1994** as an open-source project.
- PHP is a **server-side scripting language** embedded in HTML.
- It is widely used for **dynamic web content, session management, databases, and e-commerce**.
- PHP supports major databases like **MySQL, PostgreSQL, Oracle, Microsoft SQL Server**, etc.

Key Features of PHP

- **Integrated with multiple databases.**
- **Fast execution**, especially with Apache on Unix.
- Supports protocols like **POP3, IMAP, LDAP**.
- PHP is **forgiving** with errors, making it beginner-friendly.
- PHP syntax is **similar to C language**.

PHP: "Hello World" Script

- PHP is a **server-side scripting language** embedded within HTML.
- The classic **“Hello, World!”** example helps beginners understand PHP basics.
- PHP code is enclosed within **special tags**:
  - <?php ... ?> ✅ *(most commonly used)*
  - <? ... ?>
  - <script language="php"> ... </script>
- When the PHP code is executed:
  - The **server processes** the PHP.
  - Only the **HTML output** is sent to the browser.
  - **PHP code is not visible** in the page source from the browser.
- Use echo or print to output text.

Example

```
<html>
<head>
  <title>Hello World</title>
</head>
<body>

  <?php
    echo "Hello, World!";
  ?>

</body>
</html>
```

PHP does not run in a browser directly like HTML.

- Step 1: Install PHP on Windows/MacOS/Linux
  - Set PHP in System PATH
- Step 2: Create Your First PHP File(e.g, hello.php)
- Step 3: Run PHP in Browser
  - Start PHP's built-in web server: php -S localhost:8000
  - Open your browser and visit: http://localhost:8000/hello.php

PHP Markup Tags

- All PHP code must be enclosed within **special PHP tags** recognized by the PHP parser. There are **three types of PHP tags** you can use:
  - <?php ... ?> – **Most commonly used** and recommended
  - <? ... ?> – Short tag (not always enabled on all servers)
  - <script language="php"> ... </script> – Obsolete and rarely used
- The **recommended standard** for all tutorials and production code is:

```
<?php

  // Your PHP code here

?>
```

Commenting in PHP

- Comments are **non-executable** lines used to:
  - Add notes
  - Explain logic
- Comments are **ignored by the PHP parser**
- **Single-Line Comments**
  - Used for short explanations.
  - Two styles: # , //
- **Multi-Line Comment**
  - Ideal for longer explanations or pseudocode.
  - Enclosed between /* ... */
- **Multi-Line Printing (Using Here Document Syntax)**
  - Allows printing **multiple lines** easily.
  - Uses <<<END and END;
- **Multi-Line Printing (With Line Breaks in String)**
  - You can use line breaks inside strings using print or echo.

```php
<?php
// ----------------------------------------
// Single-Line Comments
// ----------------------------------------
// This is a single-line comment using slashes
# This is another single-line comment using hash
print "This is a line after single-line comments.<br>";

// ----------------------------------------
// Multi-Line Comment
// ----------------------------------------
/*
This is a multi-line comment.
Author   : Example Author
Purpose  : To demonstrate multi-line comments in PHP
Language : PHP
*/
echo "This is a line after multi-line comments.<br>";
// ----------------------------------------
// Multi-Line Printing using Heredoc
// ----------------------------------------
print <<<END
<h3>Multi-Line Output Using Heredoc:</h3>
This is line 1<br>
This is line 2<br>
This is line 3<br>
END;
echo "<br>"; // Just for spacing
// ----------------------------------------
// Multi-Line Printing Using Line Breaks in String
// ----------------------------------------
print "This is another way to print
multiple lines
in PHP using line breaks inside a string.<br>";
?>
```

Whitespace, Case Sensitivity, and Statements

- **PHP is Whitespace Insensitive**
- **Whitespace** includes: spaces, tabs, and newlines (enter key).
- PHP **ignores extra whitespace** in most places — it's used for code readability.
- You can break up code over multiple lines or include extra tabs/spaces without affecting functionality. These are all **equivalent** in PHP:
  - `$four = 2 + 2;`
  - `$four    =          2   +          2;`
  - `$four = 2          +`
  - `            2;`

Whitespace, Case Sensitivity, and Statements

- **PHP is Case Sensitive**
- **Variable names** in PHP are **case sensitive**.
- $capital, $Capital, and $CaPiTaL are all different variables.
- Function names in PHP **are not case sensitive**, but variables are.
  - Example:

```
$capital = 67;

print("Variable capital is $capital<br>");

print("Variable CaPiTaL is $CaPiTaL<br>");
```

  - **Output:**

```
Variable capital is 67

Variable CaPiTaL is                    (because $CaPiTaL is undefined)
```

Whitespace, Case Sensitivity, and Statements

- **PHP Statements End with Semicolons**
- A **statement** is any expression that performs an action or assigns a value.
- Every PHP statement must end with a **semicolon (;)**.
- PHP code blocks are formed by **a sequence of such statements** within <?php ... ?>.
  - Example:

```
$greeting = "Welcome to PHP!";
```

PHP Variables

- Variables are used to **store and manipulate data** in PHP programs.
- All variables begin with a **dollar sign ($)**.
- A variable's value is the result of its **most recent assignment**.
- PHP variables:
  - Do **not need prior declaration**.
  - Have **no fixed data type** (they are dynamically typed).
  - Can store **numbers, strings, arrays, objects**, etc.
- PHP does **automatic type conversion** when needed.
- Variables **not initialized** have default values.

PHP Variable Types

- **Simple Types :**
  - Integer
  - Double (float)
  - Boolean
  - NULL
  - String
- **Special Type :**
  - Resource (Special variables that hold references to external resources, such as database connections or file handles)
- **Compound Types:**
  - Array
  - Object

Integers

- Whole numbers (positive or negative), no decimal point.
  - Examples: 4195, -10, 0
- Can be written in:
  - **Decimal** (base 10): 123
  - **Octal** (base 8): 0123 (starts with 0)
  - **Hexadecimal** (base 16): 0x1A
- Integer range (on 32-bit systems):
  - From **-2,147,483,648 to 2,147,483,647**

Doubles (Floating Point Numbers)

- Numbers with decimals, like 3.14, 49.1
- Automatically formatted with minimal required decimal places.
- Example:

```
$many = 2.2888800;

$many_2 = 2.2111200;

$few = $many + $many_2; // Outputs: 2.28888 + 2.21112 = 4.5
```

Booleans

- Have only **two values**: true or false
- Constants TRUE and FALSE (not case-sensitive)
- Used commonly in conditions like if, while, etc.
- **Boolean Conversion Rules:**
  - **Number**: 0 is false, any other number is true.
  - **String**: "" or "0" is false, anything else is true.
  - **NULL**: Always false.
  - **Array**: [] is false, others are true.
  - **Object**: If it has a property, it's true.
  - **Resource**: Valid resources are true.

Example:

```
$true_num = 3 + 0.14159;        // true

$true_str = "Tried and true";   // true

$false_num = 999 - 999;         // false

$false_str = "";                // false

$false_null = NULL;             // false
```

NULL

- A special type with only **one value: NULL**
- Assigned by:

  $var = NULL;
- **Case-insensitive**: NULL, null, or Null work the same.
- A variable with NULL:
  - **Returns false** in a Boolean context.
  - **Returns false** when checked with isset().

Strings

- A sequence of characters (letters, numbers, symbols, etc.) can be enclosed in:
  - **Single quotes (' ')**
  - **Double quotes (" ")**
- **String Examples:**
  - $string1 = "Hello, world!";
  - $string2 = 'Single-quoted string';
  - $string3 = ""; // Empty string
- **Double vs. Single Quotes:**
  - **Single quotes**: Output literally ('$var' = $var)
  - **Double quotes**: Parse **variables**

Example:

```
$variable = "name";

$literally = 'My $variable will not print!\n'; // No parsing

print($literally);

$literally = "My $variable will print!\n"; // Parses variable

print($literally);
```

Output:

```
My $variable will not print!\n

My name will print
```

PHP Variable Naming

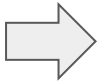
- Variable names **must begin** with a **letter (a–z or A–Z)** or an **underscore (_)**.
- After the first character, variable names may contain:
  - **Letters**
  - **Numbers**
  - **Underscores**
- **Not allowed characters**: +, -, %, (,), ., &, etc.
- There is **no length limit** on variable names.
- All PHP variables **start with a dollar sign ($)**.

PHP Variable Scope Types

- **Local Variables:**
  - Declared **inside a function**.
  - **Only accessible** within that function.
  - Same variable name **outside the function refers to a different variable**.
- **Function Parameters Variables:**
  - Declared in function definition inside `()` as inputs.
  - Behave like **local variables** within the function.
  - Can return a result using `return`.
- **Global Variables:**
  - Declared **outside** any function.
  - Can be accessed **inside a function** using the `GLOBAL` keyword.
  - Allows modification of global values within functions.
- **Static Variables**
  - Declared with the `STATIC` keyword **inside a function**.
  - Retain their **value between function calls**.
  - Useful for **counters, persistent data**, etc.

Example

```
<h2>1. Local Variable Example</h2>
<?php
$x = 4;
function assignx() {
    $x = 0; // This is a local variable, separate from the $x outside the function
    echo "\$x inside function is $x.<br>";
}
assignx();
echo "\$x outside of function is $x.<br>";
?>
<hr>
<h2>2. Function Parameter Example</h2>
<?php
function multiply($value) {
    $value = $value * 10;
    return $value;
}
$retval = multiply(10);
echo "Return value is $retval<br>";
?>
<hr>
<h2>3. Global Variable Example</h2>
<?php
$somevar = 15;
function addit() {
    GLOBAL $somevar;
    $somevar++;
    echo "Somevar is $somevar<br>";
}
addit();
?>
<hr>
<h2>4. Static Variable Example</h2>
<?php
function keep_track() {
    STATIC $count = 0;
    $count++;
    echo $count . "<br>";
}
keep_track(); // Output: 1
keep_track(); // Output: 2
keep_track(); // Output: 3
?>
```



1. Local Variable Example

$x inside function is 0.
$x outside of function is 4.

---

2. Function Parameter Example

Return value is 100

---

3. Global Variable Example

Somevar is 16

---

4. Static Variable Example

1
2
3

Operator in PHP

- An **operator** is a symbol that performs **operations** on **variables and values**.
- In the expression 4 + 5 = 9:
  - 4 and 5 are **operands**
  - + is the **operator**
- PHP supports various types of operators:
  - **Arithmetic Operators**
  - **Comparison Operators**
  - **Logical (Relational) Operators**
  - **Assignment Operators**
  - **Conditional (Ternary) Operators**

Arithmetic Operator Example

```php
<?php
$a = 42;
$b = 20;

$c = $a + $b;
echo "Addition Operation Result: $c <br/>";

$c = $a - $b;
echo "Subtraction Operation Result: $c <br/>";

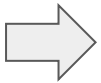
$c = $a * $b;
echo "Multiplication Operation Result: $c <br/>";

$c = $a / $b;
echo "Division Operation Result: $c <br/>";

$c = $a % $b;
echo "Modulus Operation Result: $c <br/>";

$c = $a++;
echo "Increment Operation Result (before increment): $c <br/>";

$c = $a--;
echo "Decrement Operation Result (before decrement): $c <br/>";
?>
```



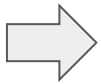
Addition Operation Result: 62
Subtraction Operation Result: 22
Multiplication Operation Result: 840
Division Operation Result: 2.1
Modulus Operation Result: 2
Increment Operation Result (before increment): 42
Decrement Operation Result (before decrement): 43

Comparison Operators Example

```php
<?php
$a = 10;
$b = 20;

echo "a = $a, b = $b<br><br>";

echo "a == b: " . var_export($a == $b, true) . "<br>";
echo "a === b: " . var_export($a === $b, true) . "<br>";
echo "a != b: " . var_export($a != $b, true) . "<br>";
echo "a <> b: " . var_export($a <> $b, true) . "<br>";
echo "a !== b: " . var_export($a !== $b, true) . "<br>";
echo "a > b: " . var_export($a > $b, true) . "<br>";
echo "a < b: " . var_export($a < $b, true) . "<br>";
echo "a >= b: " . var_export($a >= $b, true) . "<br>";
echo "a <= b: " . var_export($a <= $b, true) . "<br>";
?>
```



a = 10, b = 20

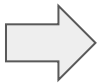
a == b: false
a === b: false
a != b: true
a <> b: true
a !== b: true
a > b: false
a < b: true
a >= b: false
a <= b: true

Logical (Relational) Operators Example

```php
<?php
$a = true;
$b = false;

echo "a = true, b = false<br><br>";

echo "a and b: " . var_export($a and $b, true) . "<br>";
echo "a or b: " . var_export($a or $b, true) . "<br>";
echo "a xor b: " . var_export($a xor $b, true) . "<br>";
echo "a && b: " . var_export($a && $b, true) . "<br>";
echo "a || b: " . var_export($a || $b, true) . "<br>";
echo "!a: " . var_export(!$a, true) . "<br>";
echo "!b: " . var_export(!$b, true) . "<br>";
?>
```



a = true, b = false

a and b: false
a or b: true
a xor b: true
a && b: false
a || b: true
!a: false
!b: true

Assignment Operators Example

```
<h2>Assignment Operators in Action</h2>
<?php
$a = 10;
echo "Initial value of a: $a <br>";

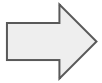
$a += 5;
echo "After a += 5: $a <br>";

$a -= 3;
echo "After a -= 3: $a <br>";

$a *= 2;
echo "After a *= 2: $a <br>";

$a /= 4;
echo "After a /= 4: $a <br>";

$a %= 3;
echo "After a %= 3: $a <br>";
?>
```

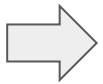


Assignment Operators in Action

Initial value of a: 10
After a += 5: 15
After a -= 3: 12
After a *= 2: 24
After a /= 4: 6
After a %= 3: 0

Conditional (Ternary) Operators Example

```php
<?php
$age = 18;
$status = ($age >= 18) ? "Adult" : "Minor";
echo "Status: $status";
?>
```



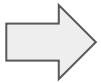
Status: Adult

PHP : Decision Making Statements

- PHP provides control flow statements to **execute different blocks of code** based on conditions. These include:
- **If...else:**
  - Used to run one block if a condition is true, and another block if it's false.
- **If...elseif...else:**
  - Used when you have **multiple conditions** to test in sequence.
- **Switch:**
  - Used to **replace long chains of if...elseif** with a cleaner, more readable alternative when testing a single variable against many values.

Example

```
<h2>1. if...else Example</h2>
<?php
$d = date("D");
if ($d == "Fri") {
    echo "Have a nice weekend!";
} else {
    echo "Have a nice day!";
}
?>
<hr>
<h2>2. if...elseif...else Example</h2>
<?php
$d = date("D");
if ($d == "Fri") {
    echo "Have a nice weekend!";
} elseif ($d == "Sun") {
    echo "Have a nice Sunday!";
} else {
    echo "Have a nice day!";
}
?>
<hr>
```



1. if...else Example

Have a nice day!

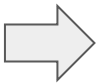
---

2. if...elseif...else Example

Have a nice day!

Example Contd…

```
<h2>1. switch Example</h2>
<?php
$d = date("D");
switch ($d) {
    case "Mon":
        echo "Today is Monday";
        break;
    case "Tue":
        echo "Today is Tuesday";
        break;
    case "Wed":
        echo "Today is Wednesday";
        break;
    case "Thu":
        echo "Today is Thursday";
        break;
    case "Fri":
        echo "Today is Friday";
        break;
    case "Sat":
        echo "Today is Saturday";
        break;
    case "Sun":
        echo "Today is Sunday";
        break;
    default:
        echo "Wonder which day is this?";
}
?>
```



1. switch Example

Today is Monday

PHP - Loop Types

- **PHP supports the following loops:**
  - **for loop** – Use when the number of iterations is known.
  - **while loop** – Use when the loop continues as long as a condition is true.
  - **do...while loop** – Executes code at least once, then checks the condition.
  - **foreach loop** – Specifically for iterating over arrays.
- **Control Statements:**
  - **break** – Exits the loop completely.
  - **continue** – Skips the current iteration and moves to the next.

Example

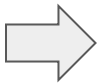
```
<h2>1. for Loop</h2>
<?php
$a = 0;
$b = 0;
for ($i = 0; $i < 5; $i++) {
    $a += 10;
    $b += 5;
}
echo "At the end of the loop a = $a and b = $b<br>";
?>

<hr>

<h2>2. while Loop</h2>
<?php
$i = 0;
$num = 50;
while ($i < 10) {
    $num--;
    $i++;
}
echo "Loop stopped at i = $i and num = $num<br>";
?>

<hr>

<h2>3. do...while Loop</h2>
<?php
$i = 0;
do {
    $i++;
} while ($i < 10);
echo "Loop stopped at i = $i<br>";
?>
```



1. for Loop

At the end of the loop a = 50 and b = 25

---

2. while Loop

Loop stopped at i = 10 and num = 40

---

3. do...while Loop

Loop stopped at i = 10

Example contd…

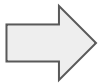
```php
<h2>1. foreach Loop</h2>
<?php
$array = array(1, 2, 3, 4, 5);
foreach ($array as $value) {
    echo "Value is $value <br>";
}
?>

<hr>

<h2>2. break Statement Example</h2>
<?php
$i = 0;
while ($i < 10) {
    $i++;
    if ($i == 3) break;
}
echo "Loop stopped at i = $i<br>";
?>

<hr>

<h2>3. continue Statement Example</h2>
<?php
$array = array(1, 2, 3, 4, 5);
foreach ($array as $value) {
    if ($value == 3) continue;
    echo "Value is $value <br>";
}
?>
```



1. foreach Loop

Value is 1
Value is 2
Value is 3
Value is 4
Value is 5

---

2. break Statement Example

Loop stopped at i = 3

---

3. continue Statement Example

Value is 1
Value is 2
Value is 4
Value is 5

PHP : Arrays

- A **data structure** that stores **multiple values** in a **single variable**.
- **Numeric Arrays**
  - Indexed by **numbers** (default: 0, 1, 2…).
  - Values can be strings, numbers, or objects.
  - Can be declared using array() or index assignment.
- **Associative Arrays**
  - Indexed using **strings** (key-value pairs).
  - Useful when the data has labels (e.g. employee => salary).
- **Multidimensional Arrays**
  - Arrays containing **other arrays**.
  - Can be accessed using **multiple indexes**.

Example

```php
<h2>1. Numeric Arrays</h2>
<?php
// First method
$numbers = array(1, 2, 3, 4, 5);
foreach ($numbers as $value) {
    echo "Value is $value <br />";
}

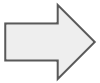
// Second method
$numbers[0] = "one";
$numbers[1] = "two";
$numbers[2] = "three";
$numbers[3] = "four";
$numbers[4] = "five";

foreach ($numbers as $value) {
    echo "Value is $value <br />";
}
?>

<hr>

<h2>2. Associative Arrays</h2>
<?php
// First method
$salaries = array("Sahil" => 2000, "Rajesh" => 1000, "zara" => 500);
echo "Salary of Sahil is " . $salaries['Sahil'] . "<br />";
echo "Salary of Rajesh is " . $salaries['Rajesh'] . "<br />";
echo "Salary of zara is " . $salaries['zara'] . "<br />";

// Second method (overwrite)
$salaries['Sahil'] = "high";
$salaries['Rajesh'] = "medium";
$salaries['zara'] = "low";
echo "Salary of Sahil is " . $salaries['Sahil'] . "<br />";
echo "Salary of Rajesh is " . $salaries['Rajesh'] . "<br />";
echo "Salary of zara is " . $salaries['zara'] . "<br />";
?>
```



1. Numeric Arrays

Value is 1
Value is 2
Value is 3
Value is 4
Value is 5
Value is one
Value is two
Value is three
Value is four
Value is five

---

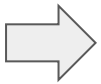
2. Associative Arrays

Salary of Sahil is 2000
Salary of Rajesh is 1000
Salary of zara is 500
Salary of Sahil is high
Salary of Rajesh is medium
Salary of zara is low

Example Contd…

```php
<h2>1. Multidimensional Arrays</h2>
<?php
$marks = array(
    "Sahil" => array(
        "physics" => 35,
        "maths" => 30,
        "chemistry" => 39
    ),
    "Rajesh" => array(
        "physics" => 30,
        "maths" => 32,
        "chemistry" => 29
    ),
    "zara" => array(
        "physics" => 31,
        "maths" => 22,
        "chemistry" => 39
    )
);

// Accessing values
echo "Marks for Sahil in physics: " . $marks['Sahil']['physics'] . "<br />";
echo "Marks for Rajesh in maths: " . $marks['Rajesh']['maths'] . "<br />";
echo "Marks for zara in chemistry: " . $marks['zara']['chemistry'] . "<br />";
?>
```



1. Multidimensional Arrays

Marks for Sahil in physics: 35
Marks for Rajesh in maths: 32
Marks for zara in chemistry: 39

PHP : Web Concepts

- **Identifying Browser & Platform**
  - PHP can access browser info using $_SERVER['HTTP_USER_AGENT'].
  - getenv() can also fetch environment variables.
  - Browser type & OS can be detected and used to personalize content.
- **Display Images Randomly**
  - Use rand() to display a different image each time the page loads.
- **Browser Redirection**
  - Use header("Location:...") to redirect to another website.
- Using HTML Forms with PHP

Identifying Browser & Platform

- PHP uses the $_SERVER['HTTP_USER_AGENT'] variable to detect the client's browser and OS.
- getenv() can also retrieve environment variables.
- preg_match() is used for matching patterns in user agent strings.

```php
<?php
$userAgent = $_SERVER['HTTP_USER_AGENT'];
echo "Your browser info: " . $userAgent;
?>
```



Your browser info: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36

Display Images Randomly

- Use rand() to generate random numbers.
- srand() seeds the random generator.
- Useful for rotating banners, ads, or visuals.

```php
<?php
$images = ["img1 (1).png", "img1 (2).png", "img1 (3).png", "img1 (4).png"]
$randIndex = rand(0, count($images) - 1);
echo "<img src='{$images[$randIndex]}' />";
?>
```

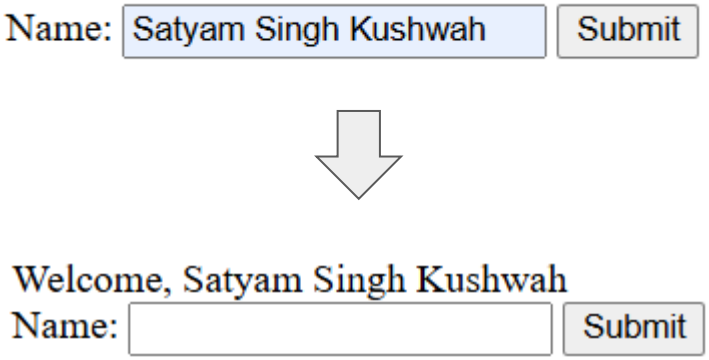
This code displays different images from an array each time the website is reloaded.

HTML Forms + PHP

- $_POST is used to capture form input when method="POST".
- Always validate input to prevent code injection.

```php
<?php
$name = "";
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  if (isset($_POST['name'])) {
    $name = htmlspecialchars($_POST['name']);
    echo "Welcome, " . $name;
  }
}
?>

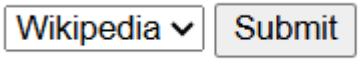
<form method="post">
  Name: <input type="text" name="name" />
  <input type="submit" />
</form>
```



Browser Redirection

- header("Location: URL") is used to redirect the browser.
- Use exit() after the header() call.

```php
<?php
// Redirect before any HTML output
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST['site'])) {
    header("Location: " . $_POST['site']);
    exit();
}
?>
<form method="post">
  <select name="site">
    <option value="https://www.google.com">Google</option>
    <option value="https://www.wikipedia.org">Wikipedia</option>
  </select>
  <input type="submit" />
</form>
```



The Wikipedia website will open in the browser.

MySQL Overview

- MySQL is an open-source relational database management system (RDBMS) software used for storing and managing data.
- It uses SQL (Structured Query Language) for managing and querying data.
- Fast, free, stable database
- Syntax is similar to Oracle
- Many of the same features as Oracle
- Frequently used in conjunction with Linux, Apache, and PHP

MySQL – Database Basics

- A relational database manager (MySQL) manages databases which holds tables which has records (rows) with attributes (columns)
- Each record must have a unique ID, also known as a Primary Key. When used as an identifier in another table it's called a Foreign Key. Used for joins.
- Each attribute has to have a data type. (e.g. int, text, varchar)
- A database language (SQL) is used to create and delete databases and manage data

MySQL – Create Tables

```
CREATE TABLE Student
(
        ID int(4) auto_increment,
                username varchar(255),
                email varchar(255),
                Address varchar(255),
                PRIMARY KEY (ID)
);
```

```
CREATE TABLE Products
(
    ProductID int(4) AUTO_INCREMENT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
    InStock INT DEFAULT 0 CHECK (InStock >= 0)
);
```

MySQL – INSERT

- Common SQL Statement: INSERT

```
INSERT INTO Student
(ID,username,email,Address)
VALUES
(12,'Rohan','rohans@iitbhilai.ac.in','Durg')
```

- Creates a new record in the table Student

MySQL - SELECT

- Common SQL Statement: SELECT

```
SELECT id, username
FROM Student
```

- Selects the attributes ‘id’ and ‘username’ from every record in ‘Student’
  - SELECT is how you query the database. You can also:
  - limit the number of records returned with LIMIT,
  - limit retrieval to those records that match a condition with WHERE,
  - sort the data after the query has been evaluated using ORDER BY

MySQL – UPDATE

- Common SQL Statement: UPDATE

```
UPDATE Student
SET email = 'sohamsk@iitbhilai.ac.in'
WHERE uid = 1
```

- Updates the email address where 'uid = 1' in the table 'student'
- In this case I know that uid 1 is what my record was. In many cases you'd pass a uid variable from a form.

MySQL - DELETE

- Common SQL Statement: DELETE

```
DELETE FROM Student
WHERE uid = 1
```

- Deletes the record where ‘uid = 1’ in the table ‘oscarpool’
- DELETE only removes the record from the table. To remove an entire table from the database you need to use the SQL statement DROP.

MySQL - Database Access

```
<?php

$conn = mysqli_connect("localhost", "root", "", "school");

if (!$conn) {

    die("Connection failed: " . mysqli_connect_error());

}

echo "Connected successfully";

?>
```

Parameter	Meaning
`localhost`	Host (usually local PC)
`root`	Default username
`""`	Default password (empty)
`"school"`	Your database name
`mysqli_connect()`	Connects PHP to MySQL

Handling Query Example

```
<?php

$conn = mysqli_connect("localhost", "root", "", "school");

$result = mysqli_query($conn, "SELECT * FROM students");

while ($row = mysqli_fetch_assoc($result)) {

    echo "Name: " . $row["name"] . "<br>";

}

mysql_close($conn);

?>
```

Common Errors

Error	Reason
`Access denied`	Wrong username/password
`Unknown database`	Database name misspelled
`mysqli_connect(): failed`	MySQL server not running
Blank page	No error handling or `display_errors` is off

Insert Data into MySQL

```
<?php

$conn = mysqli_connect("localhost", "root", "", "school");

$sql = "INSERT INTO students (name, age) VALUES (‘John’, 20)";

if (mysqli_query($conn, $sql)) {

    echo "New record inserted successfully!";

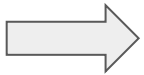
} else {

    echo "Error: " . mysql_error($conn);

}

mysql_close($conn);

?>
```

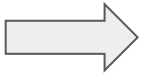


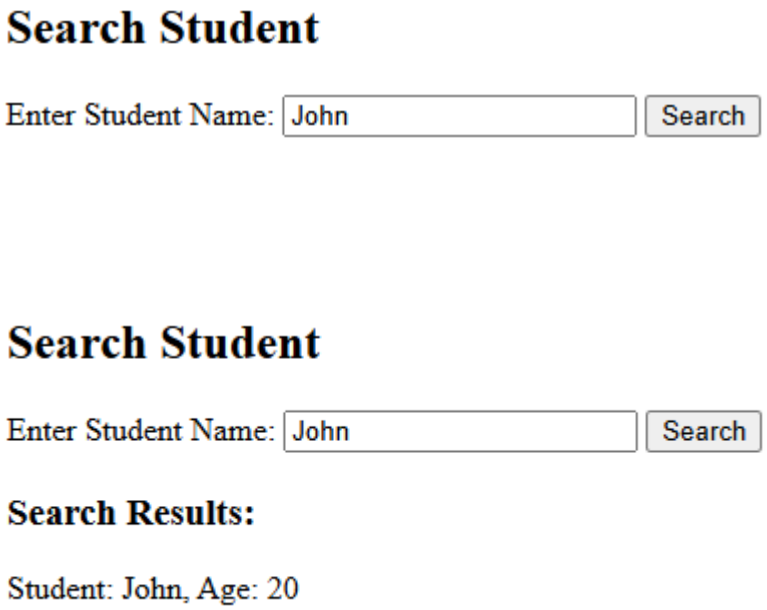
New record inserted successfully!

name	age
John	20

Fetch Data with User Input

```
<?php
$name = $_GET['name'];
$conn = mysqli_connect("localhost", "root", "", "school");
$sql = "SELECT * FROM students WHERE name='$name'";
$result = mysqli_query($conn, $sql);
while ($row = mysqli_fetch_assoc($result)) {
    echo "Student: " . $row['name'] . ", Age: " . $row['age'] . "<br>";
}
?>
```





Update Data

```
<?php

$conn = mysqli_connect("localhost", "root", "", "school");

$sql = "UPDATE students SET age = 21 WHERE name =
‘Rohan’”;

if (mysqli_query($conn, $sql)) {

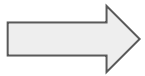
    echo "Record updated successfully!";

} else {

    echo "Error: " . mysqli_error($conn);

}

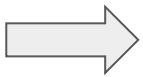
?>
```



Record updated successfully!

Delete Record

```
<?php
$conn = mysqli_connect("localhost", "root", "", "school");
$sql = "DELETE FROM students WHERE name = “John”;
if (mysqli_query($conn, $sql)) {
    echo "Record deleted!";
} else {
    echo "Error deleting: " . mysqli_error($conn);
}
?>
```



Record deleted!

Display Data in HTML Table

```
<?php

$conn = mysqli_connect("localhost", "root", "", "school");

$result = mysqli_query($conn, "SELECT * FROM students");

echo "<table
border='1'><tr><th>Name</th><th>Age</th></tr>";

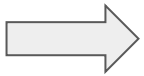
while ($row = mysqli_fetch_assoc($result)) {

    echo "<tr><td>" . $row['name'] . "</td><td>" . $row['age'] .
"</td></tr>";

}

echo "</table>";

?>
```



Name	Age
John	20